SEGPRD – PL

Diogo Silva - 1211882

Hugo Ribeiro - 1250520

António Azevedo - 1250494

Gonçalo Carrusca - 1200882

PL1

Exercício 1

1.1

No exercício 1.1 foi utilizada a ferramenta OpenSSL em linha de comandos para testar o funcionamento do algoritmo DES. Comecei por criar uma mensagem simples em texto plano com o comando echo "Mensagem secreta SEGPRD" > mensagem.txt. De seguida, defini a chave "segredo" e utilizei-a para cifrar a mensagem através do comando openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -in mensagem.txt -out mensagem.enc -k segredo -base64, obtendo assim um ficheiro cifrado em formato Base64.

Posteriormente, procedi à decifragem do ficheiro com a mesma chave usando openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -d -in mensagem.enc -out mensagem_dec.txt -k segredo -base64, o que permitiu recuperar corretamente a mensagem original. Durante este processo ocorreu inicialmente um erro (bad decrypt), que foi identificado como sendo causado pela utilização de parâmetros diferentes entre a cifragem e a decifragem, nomeadamente a presença de -pbkdf2 apenas num dos comandos. Após corrigir esta inconsistência e garantir que ambos os comandos utilizavam exatamente os mesmos parâmetros, a decifragem passou a funcionar corretamente.

Por fim, foi testada a decifragem com uma chave incorreta através do comando openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -d -in mensagem.enc -out tentativa.txt -k chaveerrada -base64, tendo como resultado um erro ou texto ilegível. Este comportamento confirma que, em criptografia simétrica, é essencial utilizar a mesma chave e os mesmos parâmetros para conseguir recuperar a mensagem original.

```
U2FsdGVkX1/iEFT+h/KJghEQdLcub+wvvXeWNpiy3qzhpUMSSxXZnpE3XsuKBtRn
diogo@Second:~/SEGPRD$ openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -d -in mensagem.enc -out mensagem
_dec.txt -k segredo -base64
diogo@Second:~/SEGPRD$ cat mensagem_dec.txt
Mensagem secreta SEGPRD
diogo@Second:~/SEGPRD$
```

```
diogo@Second:~$ cd SEGPRD/
diogo@Second:~/SEGPRD$ openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -d -in mensagem.enc -out tentativ
a.txt -k chaveerrada -base64
bad decrypt
406793F84B760000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:../providers/implementations/cipher
s/ciphercommon_block.c:124:
```

1.2

No exercício 1.2 foi proposto realizar um ataque de brute force a um ciphertext fornecido (s8knpxKRpJ3JDd+UJqCupw==), com o objetivo de descobrir a chave utilizada.

Para resolver este problema, a abordagem definida consistiu em testar automaticamente um grande conjunto de possíveis chaves até encontrar uma que permitisse decifrar corretamente o conteúdo.

Inicialmente, foi preparada uma wordlist, nomeadamente o ficheiro rockyou.txt, que contém um conjunto extenso de palavras-passe comuns. Em seguida, foi desenvolvido um pequeno script em Python que, para cada entrada da wordlist, executava o comando OpenSSL para tentar decifrar o ciphertext. A lógica passava por iterar sobre cada possível chave e verificar se o resultado da decifragem não gerava erro (bad decrypt) ou produzia um output legível.

O comando utilizado em cada tentativa seguia a estrutura:

openssl enc -provider default -provider legacy -des-ecb -pbkdf2 -d -base64 -k <chave>

Este processo permitiria, em teoria, encontrar a chave correta caso esta estivesse presente na wordlist ou dentro do espaço de procura considerado.

No entanto, durante a execução do exercício, foi identificado (e posteriormente confirmado pelo docente) que o ciphertext fornecido apresenta um erro, tornando a sua decifragem por brute force inviável. Como consequência, mesmo após múltiplas tentativas com diferentes chaves da wordlist, não foi possível obter um resultado válido.

Assim, após validar que o procedimento estava corretamente implementado e que não existiam erros na abordagem utilizada, a tentativa de brute force foi interrompida, uma vez que o problema não era resolúvel nas condições fornecidas.

Exercício 2

O que é o AES e porque é usado

O AES (Advanced Encryption Standard) é um algoritmo de encriptação simétrica por blocos, definido como padrão pelo NIST em 2001 para substituir o DES. Sendo simétrico, utiliza a mesma chave tanto para encriptar como para desencriptar dados.

Conceitos técnicos utilizados

- **AES-256**: versão do AES com chave de 256 bits, oferecendo um nível de segurança muito elevado.
- **Modo CBC (Cipher Block Chaining)**: cada bloco de dados depende do bloco anterior, evitando padrões repetidos no resultado final.
- **IV (Initialization Vector)**: valor aleatório usado no início da encriptação, garantindo que a mesma mensagem nunca gera o mesmo resultado.
- **Padding**: preenchimento do último bloco, já que o AES trabalha sempre com blocos de 16 bytes.
- **get_random_bytes**: função que gera valores aleatórios seguros; se a chave for previsível, toda a segurança fica comprometida.

Comparação: AES vs DES vs 3DES

- **DES**: usa chave de 56 bits, atualmente considerada insegura, pois pode ser quebrada rapidamente com força bruta.
- **3DES**: aplica o DES três vezes, aumentando a segurança, mas tornando o processo muito mais lento e ainda vulnerável a certos ataques.
- **AES-256**: mais moderno, seguro e eficiente, com chaves maiores e melhor desempenho.

Conclusão

```python
def aes_encrypt(input_path, output_path, key, iv):
    """Encripta um ficheiro usando AES-256 em modo CBC."""
    with open(input_path, 'rb') as f:
        data = f.read()

    cipher = AES.new(key, AES.MODE_CBC, iv)
    encrypted_data = cipher.encrypt(pad(data, AES.block_size))

    with open(output_path, 'wb') as f:
        f.write(encrypted_data)

    print(f"[AES] Ficheiro encriptado: {output_path}")
    print(f"      Tamanho original:    {len(data)} bytes")
    print(f"      Tamanho encriptado:  {len(encrypted_data)} bytes")
```

```python
def aes_decrypt(input_path, output_path, key, iv):
    """Desencripta um ficheiro usando AES-256 em modo CBC."""
    with open(input_path, 'rb') as f:
        encrypted_data = f.read()

    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_data = unpad(cipher.decrypt(encrypted_data), AES.block_size)

    with open(output_path, 'wb') as f:
        f.write(decrypted_data)

    print(f"[AES] Ficheiro desencriptado: {output_path}")
```

```python
des_k = get_random_bytes(8)    # 56 bits efetivos
des_i = get_random_bytes(8)

start = time.perf_counter()
des_enc = des_encrypt(test_data, des_k, des_i)
des_enc_time = time.perf_counter() - start

start = time.perf_counter()
des_decrypt(des_enc, des_k, des_i)
des_dec_time = time.perf_counter() - start

results['DES'] = {
    'key_bits': 56,
    'block_bits': 64,
    'enc_time': des_enc_time,
    'dec_time': des_dec_time,
    'status': 'INSEGURO (obsoleto)'
}
```

```python
des3_k = DES3.adjust_key_parity(get_random_bytes(24))  # 168 bits
des3_i = get_random_bytes(8)

start = time.perf_counter()
des3_enc = des3_encrypt(test_data, des3_k, des3_i)
des3_enc_time = time.perf_counter() - start

start = time.perf_counter()
des3_decrypt(des3_enc, des3_k, des3_i)
des3_dec_time = time.perf_counter() - start

results['3DES'] = {
    'key_bits': 168,
    'block_bits': 64,
    'enc_time': des3_enc_time,
    'dec_time': des3_dec_time,
    'status': 'Fraco (deprecated)'
}
```

documento_secreto.txt	08-04-2026 19:08	Documento de Te...	31 KB
documento_secreto_desencriptado.txt	08-04-2026 19:27	Documento de Te...	31 KB
documento_secreto_encriptado.bin	08-04-2026 19:27	Ficheiro BIN	31 KB

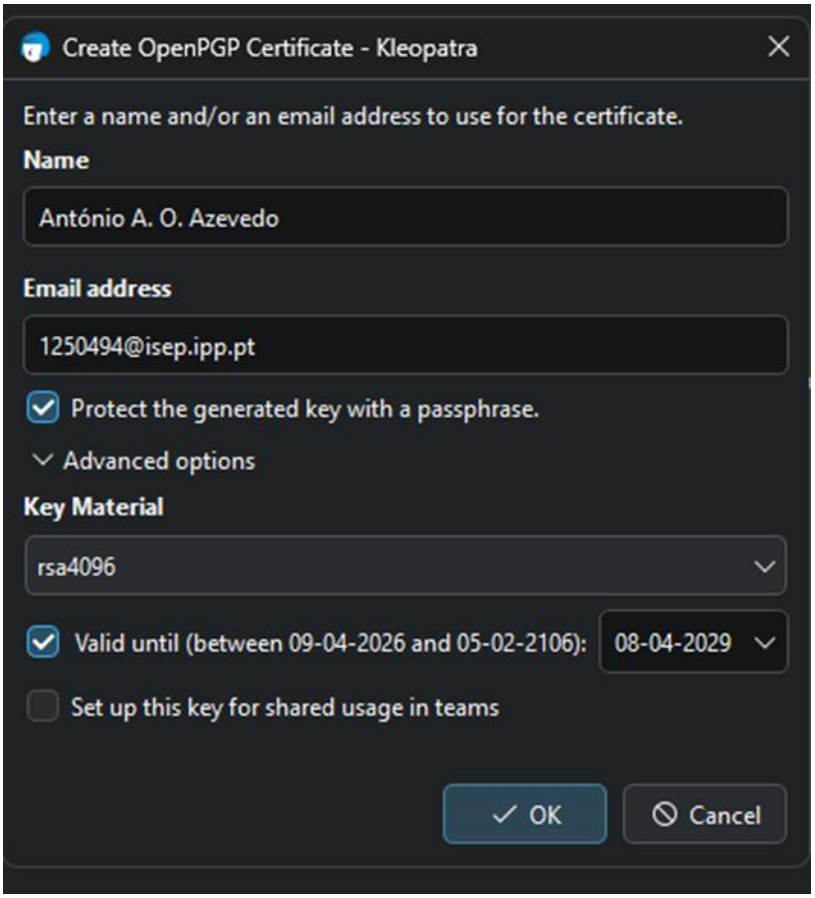
```
TAREFA 1: Encriptação AES
[AES] Ficheiro encriptado: C:\projetos\isep\dados\documento_secreto_encriptado.bin
      Tamanho original:    31000 bytes
      Tamanho encriptado:  31008 bytes
      Tempo de encriptação: 0.002470 segundos

TAREFA 2: Desencriptação AES
[AES] Ficheiro desencriptado: C:\projetos\isep\dados\documento_secreto_desencriptado.txt
      Tempo de desencriptação: 0.012337 segundos

 VERIFICAÇÃO DE INTEGRIDADE
Ficheiro recuperado é IDÊNTICO ao original

TAREFA 3 e 4: Comparação AES vs DES vs 3DES

Tamanho dos dados de teste: 10.0 MB

============================================================
Algoritmo       Chave    Bloco  Encript.(s)  Decript.(s)  Estado
------------------------------------------------------------------
AES-256         256b     128b       0.0201       0.0199  Seguro (padrão atual)
DES              56b      64b       0.1099       0.0953  INSEGURO (obsoleto)
3DES            168b      64b       0.3047       0.3017  Fraco (deprecated)
============================================================

Velocidade relativa (AES como referência = 1.0x):
  AES-256 : 1.00x mais rápido que AES-256
  DES     : 5.13x mais lento que AES-256
  3DES    : 15.15x mais lento que AES-256
```

Tendo em conta os resultados e o método de encriptação/desencriptação de cada um concluímos:

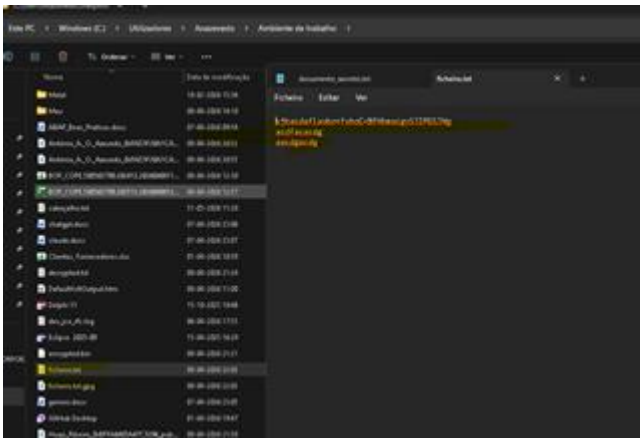
- O DES está completamente ultrapassado devido à sua baixa segurança.
- O 3DES melhora esse aspeto, mas é lento e não é recomendado.
- O AES-256 é atualmente a melhor opção: combina segurança com desempenho e foi desenhado para ser o padrão global de encriptação.

PL2

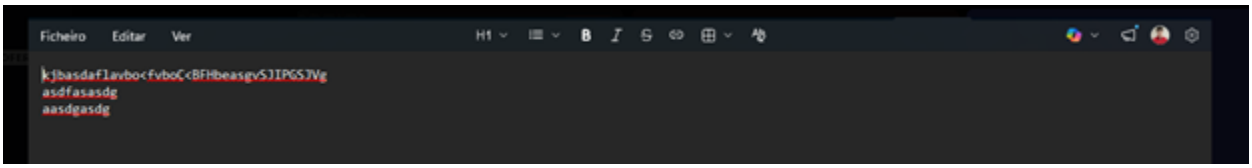
Exercício 1



Ficheiro criado por mim e enviado:



Colega depois desencriptar:



O PGP usa **criptografia assimétrica**, onde são geradas duas chaves matematicamente relacionadas:

- **Chave Pública** → partilhamos com toda a gente. Serve para outros **encriptarem mensagens para nós** e para **verificarem a nossa assinatura**
- **Chave Privada** → guardamos em segredo absoluto. Serve para **desencriptares mensagens** recebidas e para **assinares** o que enviamos

O tamanho da chave determina a resistência a ataques de força bruta e **4096 bits** oferece margem de segurança adicional para o futuro.

A passphrase protege a chave privada localmente — mesmo que alguém aceda ao nosso ficheiro de chaves, não consegue usá-la sem a passphrase.

Um **Key Server** é um repositório público onde as pessoas publicam as suas chaves públicas. Funciona como uma "lista telefónica" da criptografia — qualquer pessoa que queira enviar-nos uma mensagem encriptada pode pesquisar o nosso email e obter a nossa chave pública automaticamente.

Porquê publicar a chave?

- Facilita a distribuição da chave pública

- Permite que terceiros verifiquem a autenticidade da chave

Integra-se com a **Web of Trust** do PGP — um modelo descentralizado de confiança onde pessoas assinam as chaves umas das outras

Exercício 2

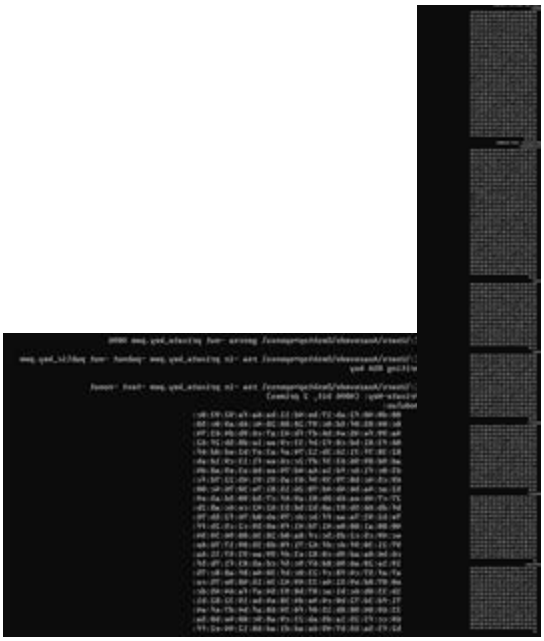
1. Fundamentos do RSA

O RSA é um algoritmo de criptografia assimétrica criado em 1977. A sua segurança baseia-se na dificuldade prática de fatorizar números muito grandes.

Funciona com duas chaves diferentes mas relacionadas:

- **Chave pública**: usada para cifrar e pode ser partilhada
- **Chave privada**: usada para decifrar e deve ser mantida secreta

O que é cifrado com uma só pode ser decifrado com a outra.



2. Geração das chaves

Ao gerar um par RSA-4096 com OpenSSL, o processo faz o seguinte:

- Cria dois números primos grandes (p e q)
- Calcula o módulo n = p × q

- Calcula $\phi(n) = (p-1) \times (q-1)$
- Define o expoente público (normalmente 65537)
- Calcula o expoente privado d

No final, apenas ficam guardadas as chaves:

- Pública → (n, e)
- Privada → (n, d)

Os valores intermédios são descartados por segurança.

A integração do PGP com o Thunderbird permite que o processo de encriptação/desencriptação seja **transparente para o utilizador** — o software trata de tudo automaticamente ao enviar e receber emails.

O Thunderbird usa o standard **OpenPGP (RFC 4880)**, que é a implementação open-source do PGP original. Quando enviamos um email encriptado:

1. O Thunderbird obtém a chave pública do destinatário
2. Encripta o conteúdo com essa chave
3. Assina digitalmente com a nossa chave privada
4. O destinatário recebe, desencripta com a sua chave privada e verifica a nossa assinatura

Encriptação + Assinatura — dois objetivos distintos:

Mecanismo	Objetivo	Como funciona
Encriptação	Confidencialidade	Usa a chave pública do professor para encriptar — só ele consegue ler
Assinatura Digital	Autenticidade + Integridade	Usa a nossa chave privada para assinar — o professor confirma que fomos nós a enviar e que a mensagem não foi alterada

Fluxo técnico:

1. O nosso Thunderbird gera um **hash** (SHA-256) do conteúdo da mensagem
2. Esse hash é encriptado com a **nossa chave privada** → isto é a assinatura digital
3. O conteúdo da mensagem é encriptado com a **chave pública de outro colega**
4. O outro colega recebe, desencripta com a sua chave privada e verifica a assinatura com a nossa chave pública

Isto garante os três pilares fundamentais de segurança: **Confidencialidade, Integridade e Autenticidade**.

Encriptação de ficheiros vs. emails:

O princípio é exatamente o mesmo, mas aplicado a ficheiros. O PGP usa um modelo híbrido:

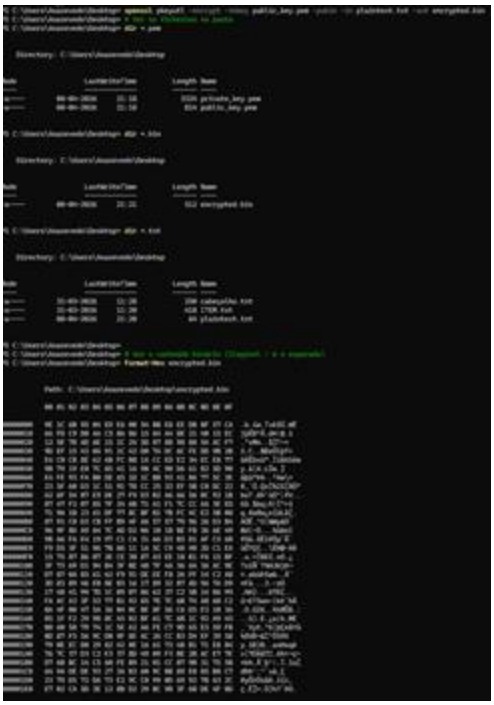
1. Gera uma **chave simétrica aleatória** (session key) — por exemplo AES-256
2. Encripta o **ficheiro com essa chave simétrica** (rápido, eficiente para grandes ficheiros)
3. Encripta a **session key com a chave pública do destinatário** (RSA)
4. O ficheiro final contém ambos: conteúdo encriptado + session key encriptada

Porquê este modelo híbrido? RSA é computacionalmente pesado para grandes volumes de dados. AES é muito mais rápido. O PGP combina o melhor dos dois mundos — **segurança da criptografia assimétrica** com a **eficiência da criptografia simétrica**.

Processo de desencriptação:

1. O GPG lê o ficheiro .gpg e identifica para quem foi encriptado (pelo Key ID)
2. Usa a **nossa chave privada** para desencriptar a session key AES
3. Usa a session key para desencriptar o conteúdo do ficheiro
4. Se houver assinatura, verifica-a usando a **chave pública do remetente**

Verificação da integridade: O GPG recalcula o hash do conteúdo desencriptado e compara com o hash que estava na assinatura. Se coincidirem → ficheiro íntegro e autêntico. Se não coincidirem → o ficheiro foi alterado ou a assinatura é inválida.



3. Cifra e decifra

O funcionamento base é:

- Cifrar: usa a chave pública
- Decifrar: usa a chave privada

Na prática, o OpenSSL usa **OAEP**, que adiciona aleatoriedade à mensagem. Isto impede que a mesma mensagem produza sempre o mesmo resultado cifrado.

O RSA tem uma limitação importante: não é feito para cifrar grandes volumes de dados diretamente.

```
PS C:\Users\Aoazevedo\Desktop> openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted.bin -out decrypted.txt
PS C:\Users\Aoazevedo\Desktop> # Ver o ficheiro decifrado
PS C:\Users\Aoazevedo\Desktop> cat decrypted.txt
Este e o meu ficheiro secreto
PS C:\Users\Aoazevedo\Desktop>
PS C:\Users\Aoazevedo\Desktop> # Comparar com o original
PS C:\Users\Aoazevedo\Desktop> fc plaintext.txt decrypted.txt
```

```
PS C:\Users\Aoazevedo\Desktop> (Get-Content plaintext.txt) -eq (Get-Content decrypted.txt)
True
```

4. Segurança e força do RSA

Quanto maior a chave, mais difícil é quebrar:

- RSA-512 → quebrável rapidamente
- RSA-1024 → já não seguro
- RSA-2048 → ainda seguro, mas com margem reduzida
- RSA-4096 → extremamente seguro com tecnologia atual

No futuro, computadores quânticos podem mudar este cenário, razão pela qual já se investiga criptografia pós-quântica.

5. Importância da entropia

A segurança do RSA depende muito da qualidade da aleatoriedade usada na geração das chaves.

Se os números primos não forem realmente aleatórios, a chave pode ser comprometida.

Um caso real mostrou isso: vários dispositivos geraram chaves com pouca entropia e acabaram por partilhar fatores primos. Bastou calcular o máximo divisor comum entre chaves públicas para as quebrar rapidamente.

Fontes de aleatoriedade:

- /dev/random → mais seguro, mas pode bloquear
- /dev/urandom → rápido e adequado para produção
- Hardware (RDRAND, TPM) → melhor qualidade

Conclusão

O RSA continua a ser um elemento essencial da segurança moderna (TLS, SSH, assinaturas digitais).

No entanto, não basta usar o algoritmo — é obrigatório:

- Usar chaves suficientemente grandes (mínimo 2048, ideal 4096)
- Garantir boa entropia na geração
- Aplicar corretamente padding seguro (como OAEP)

Se algum destes pontos falhar, toda a segurança pode ser comprometida.

Exercício 3

Neste exercício, começou-se por analisar os hashes fornecidos no enunciado:

- 93319a4441fae08309390fd2e8326002
- 543b24ecc36f9811f3c6f0ee0e6df6dbe29891bb

A identificação foi feita com base no comprimento de cada valor hexadecimal. O primeiro hash, com 32 caracteres, foi identificado como **MD5**, enquanto o segundo, com 40 caracteres, foi identificado como **SHA-1**. As funções de hash são unidirecionais, pelo que não podem ser revertidas diretamente. Na prática, a recuperação do valor original é feita de forma indireta, recorrendo a ataques de força bruta, dicionário ou bases de dados de hashes conhecidos.

Para a fase prática, foram utilizadas as ferramentas **Hashcat** e **John the Ripper** para realizar ataques de dicionário sobre os hashes fornecidos. Numa primeira fase, foi utilizada a wordlist **rockyou.txt**, por ser uma das listas de palavras-passe mais conhecidas e frequentemente usada em testes de segurança. No entanto, após a execução dos testes com ambas as ferramentas, não foi encontrada qualquer correspondência para os hashes fornecidos.

De seguida, foi utilizada uma wordlist alternativa da **CrackStation**, composta por passwords reais recolhidas de fugas de dados públicas. Esta abordagem revelou-se mais eficaz, uma vez que permitiu aumentar significativamente a probabilidade de encontrar correspondências. Para o primeiro hash, identificado como MD5, foi utilizado o **Hashcat** em modo de ataque de dicionário (-m 0), tendo sido possível recuperar a palavra-passe original **"dei"**. Para o segundo hash, identificado como SHA-1, foi utilizado o modo **100**, permitindo recuperar a palavra-passe **"isep"**. Posteriormente, os mesmos testes foram repetidos com o **John the Ripper**, recorrendo aos formatos **raw-md5** e **raw-sha1**, utilizando a mesma wordlist da CrackStation, tendo sido obtidos os mesmos resultados.

Relativamente às colisões, estas ocorrem quando duas entradas diferentes produzem exatamente o mesmo valor de hash. Isto acontece porque as funções de hash mapeiam um conjunto de entradas muito maior para um espaço de saída fixo e limitado, tornando inevitável a existência de colisões. Este problema é especialmente relevante em algoritmos antigos como **MD5** e **SHA-1**, que apresentam vulnerabilidades conhecidas e já não são recomendados para aplicações modernas de segurança.

Exercício 4

1. Imagem 1 - ex4
2. Uma colisão ocorre quando dois diferentes inputs na mesma função produzem a mesma hash. Como no artigo, referente ao SHAttered attack, que relata a primeira colisão num algoritmo SHA-1. Os investigadores conseguiram com que dois pdf's, com conteúdos diferentes, que gerassem exatamente a mesma hash SHA-1. Este ataque demonstrou que o algoritmo já não garantia a não colisão. Outro estudo, também verificou que Sha-1 já não era segura e que devia-se migrar para SHA-256 ou SHA-512, porque demonstram maior capacidade de resistência a colisões.
3.

Feature	**SHA-256**	**SHA-512**
Hash size	256 bits	512 bits
Output length	64 hex characters	128 hex characters
Security level	Very strong	Stronger
Collision resistance	$\sim 2^{128}$	$\sim 2^{256}$
Performance	Faster on 32-bit systems	Faster on 64-bit systems

4. Imagem 2 – ex4
5. Pelo artigo, podemos verificar que gerar hashes sha512 para inúmeros ficheiros pode demorar muito tempo, especialmente numa pesquisa por um diretório grande ou um file system. Isto acontece, porque para fazer o checksum do ficheiro, é necessário verificar todo o seu conteúdo. Ferramentas como o AIDE são para monitorizar a integridade de um sistema, criando uma base de dados com hashes de ficheiros relevantes. Esta ferramenta recalcula, periodicamente a hash e compara-a com valores prévios. Se houver uma modificação, a ferramenta detecta uma alteração não autorizada. Este processo pode ser dispendioso se for num diretório com muitos ficheiros, pelo que, se deve selecionar os críticos para não ter

de recalcular ficheiros em excesso, causando uma sobrecarga excessiva no sistema.

PL3

Exercício 1

O Cartão de Cidadão (CC) e a Chave Móvel Digital (CMD) são dois mecanismos oficiais de autenticação e assinatura digital em Portugal. O CC apresenta um nível de segurança mais elevado, uma vez que utiliza um certificado armazenado fisicamente no chip do cartão e exige a introdução de PIN, reduzindo o risco de uso indevido. No entanto, a sua utilização depende de um leitor de cartões e da instalação do software Autenticação.gov, o que torna o processo menos prático.

Por outro lado, a CMD oferece maior comodidade e acessibilidade, pois permite autenticação e assinatura através do telemóvel, sem necessidade de hardware adicional. Apesar de também ser segura, por depender de fatores como SMS ou aplicação móvel, pode estar mais sujeita a problemas associados ao dispositivo ou à rede.

Em conclusão, o CC oferece maior robustez e segurança, enquanto a CMD se destaca pela facilidade de utilização e disponibilidade no dia a dia.

[PL3->Exercício1](verificar documento assinado e email assinado na pasta)

Exercício 2

1. openssl req -x509 -nodes -days 3650 -newkey rsa:2048 -keyout server.key -out server.crt + Imagem 1
1. sudo apt update
   sudo apt install apache2
2. sudo a2enmod ssl

sudo systemctl restart apache2

3. sudo nano /etc/apache2/sites-available/default-ssl.conf

   configurar server name localhost e adicionar os certificados criados no 1

4. sudo nano /etc/apache2/sites-available/000-default.conf

   fazer redirect para o https e por server name localhost

5. Strict-Transport-Security (HSTS)
   Obriga o browser a usar sempre HTTPS.
   Evita ataques de downgrade (HTTP inseguro).

   Content-Security-Policy (CSP)
   Define de onde podem ser carregados scripts, imagens, etc.
   Protege contra ataques XSS e código malicioso.

   X-Content-Type-Options (nosniff)
   Impede o browser de adivinhar tipos de ficheiro.
   Evita execução de ficheiros maliciosos.

   X-Frame-Options
   Impede que o site seja carregado dentro de um iframe.
   Protege contra clickjacking.

   X-XSS-Protection
   Ativa proteção básica contra XSS no browser.
   Bloqueia scripts suspeitos (embora seja legacy).

Parte 2

1. sudo apt install nginx

2. sudo systemctl start nginx
   sudo systemctl enable nginx

3. sudo mkdir /var/www/site443
   sudo mkdir /var/www/site8080

4. echo "Site 443" | sudo tee /var/www/site443/index.html
   echo "Site 8080" | sudo tee /var/www/site8080/index.html

5. openssl req -x509 -nodes -days 3650 -newkey rsa:2048 -keyout site443.key -out site443.crt

6. openssl req -x509 -nodes -days 3650 -newkey rsa:2048 -keyout site8080.key -out site8080.crt

7. sudo nano /etc/nginx/sites-available/site443

8. configurar:
   listen 443 sslroot /var/www/site443
   ssl_certificate site443.crt
   ssl_certificate_key site443.key

9. sudo nano /etc/nginx/sites-available/site8080

10. configurar:
    listen 8080 ssl
    root /var/www/site8080
    ssl_certificate site8080.crt
    ssl_certificate_key site8080.key

11. sudo ln -s /etc/nginx/sites-available/site443 /etc/nginx/sites-enabled/
    sudo ln -s /etc/nginx/sites-available/site8080 /etc/nginx/sites-enabled/

12. sudo nginx -t
    sudo systemctl restart nginx

Diferenças entre Apache e Nginx

Apache utiliza módulos e comandos como a2enmod e a2ensite.
Nginx utiliza ficheiros de configuração com blocos server e links simbólicos.
No Apache o SSL é ativado por módulos, enquanto no Nginx é definido diretamente na configuração.

Exercício 3

A mensagem termina com =, que é um indicador claro de **Base64 encoding**.

KkBGIEQ5QEY9NSBERUY1SiA3QEMgRTk2IDZJMj5dIHNAP1ZFIDdA

Qzg2RSBFOTJFIEpARiA5Mkc2IDI/IDJERDo4Pz42P0UgRUAgNUA=

descodificada:
*@F D9@F=5 DEF5J 7@C E96 6I2>] s@?VE 7@C86E E92E J@F 92G6 2? 2DD:8?>6?E E@ 5@

rot13:
*@S Q9@S=5 QRS5W 7@P R96 6V2>] f@?IR 7@P86R R92R W@S 92T6 2? 2QQ:8?>6?R R@ 5@

rot5:
*@F D4@F=0 DEF0J 2@C E41 1I7>] s@?VE 2@C31E E47E J@F 47G1 7? 7DD:3?>1?E E@ 0@

ROT18:

*@S Q4@S=0 QRS0W 2@P R41 1V7>] f@?IR 2@P31R R47R W@S 47T1 7? 7QQ:3?>1?R R@ 0@

rot24:

*@D B4@D=0 BCD0H 2@A C41 1G7>] q@?TC 2@A31C C47C H@D 47E1 7? 7BB:3?>1?C C@ 0@

ROT47:
You should study for the exam. Don't forget that you have an assignment to do

PL4

Exercício 1

1.1

Criar certificado root

- cd ~/Desktop/rootCA
- openssl genrsa -out rootCA.key 4096
- openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -out rootCA.crt

Criar certificado intermediate

- cd ../intermediateCA
- openssl genrsa -out intermediate.key 4096
- openssl req -new -key intermediate.key -out intermediate.csr

Assinar o intermediate com recurso a um ficheiro de configuração

- openssl x509 -req -in intermediate.csr \
  -CA ../rootCA/rootCA.crt -CAkey ../rootCA/rootCA.key -CAcreateserial -out intermediate.crt -days 1825 -sha256 -extfile v3_ext.cnf

Ficheiro de configuração v3_ext.conf

- basicConstraints = critical, CA:TRUE, pathlen:0

  keyUsage = critical, digitalSignature, cRLSign, keyCertSign

  subjectKeyIdentifier = hash

  authorityKeyIdentifier = keyid:always,issuer

Criar certificado Web Server

- mkdir server cd server
- openssl genrsa -out server.key 2048
- openssl req -new -key server.key -out server.csr

Assinar o certificado do server

- openssl x509 -req -in server.csr -CA ../intermediateCA/intermediate.crt -CAkey ../intermediateCA/intermediate.key -CAcreateserial -out server.crt -days 825 -sha256

Criar a cadeia de certificados

- cat server.crt ../intermediateCA/intermediate.crt > chain.crt

Validação da cadeia

- openssl verify -CAfile ../rootCA/rootCA.crt
  untrusted ../intermediateCA/intermediate.crt server.crt

1.2 Configuração do Web Server (Apache com HTTPS)

Ativar SSL

- sudo a2enmod ssl
  sudo systemctl restart apache2

Criar diretório e copiar certificados

- sudo mkdir -p /etc/apache2/certs
  sudo cp ~/Desktop/server/server.key /etc/apache2/certs/
  sudo cp ~/Desktop/server/server.crt /etc/apache2/certs/
  sudo cp ~/Desktop/intermediateCA/intermediate.crt /etc/apache2/certs/

Criar FullChain

- cat ~/Desktop/server/server.crt ~/Desktop/intermediateCA/intermediate.crt | sudo tee /etc/apache2/certs/fullchain.crt

Editar Virtual Host SSL – default-ssl.conf

- SSLEngine on
  SSLCertificateFile /etc/apache2/certs/fullchain.crt
  SSLCertificateKeyFile /etc/apache2/certs/server.key

Erros encontrados

- Porta já em uso, pois inicialmente ia usar o nginx, só que vai ficar deprecated, então continuiei no apache, possíveis improves seria usar o traefik
- Certificado não confiável, não tinha estabelecido o SAN (Subject Alternative Name), editei o ficheiro de conf do server para adicionar o alt_name = localhost, regenerei o certificado

```
openssl x509 -req -in server.csr \
-CA ../intermediateCA/intermediate.crt \
-CAkey ../intermediateCA/intermediate.key \
-CAcreateserial \
-out server.crt -days 825 -sha256 \
-extfile v3_ext.cnf
```

Teste com curl

- curl -k https://localhost
- Resultado : Página degault html apache

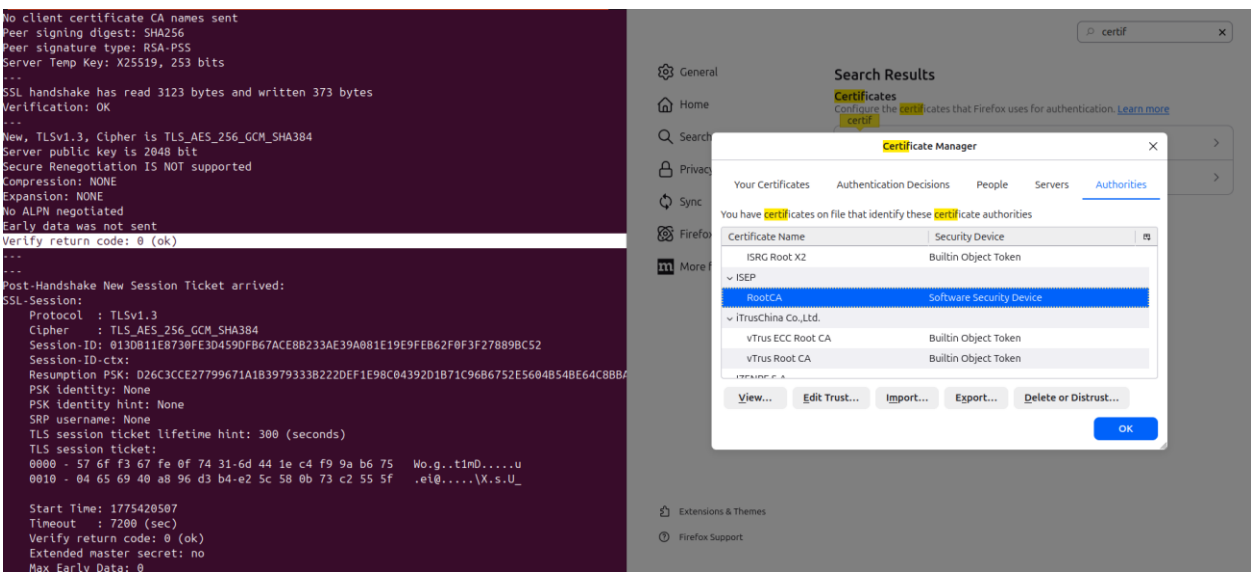
```
deus@Ubuntu-Lab:~/Desktop/server$ curl -k https://localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xh
tml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

    font-family: Ubuntu, Verdana, sans-serif;
    font-size: 11pt;
    text-align: center;
  }

  div.main_page {
    position: relative;
    display: table;

    width: 800px;

    margin-bottom: 3px;
    margin-left: auto;
    margin-right: auto;
```

Teste com openssl

- openssl s_client -connect localhost:443 \
  -CAfile ~/Desktop/rootCA/rootCA.crt

```
SSL handshake has read 3123 bytes and written 373 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
---
Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
    Session-ID: 193B68430C036EC73D368951ED0D3799109D3ED5F9E4EA144B1BBD26D4AD44CE
    Session-ID-ctx:
    Resumption PSK: 79819F727025EB7BB07119AC6A88BE03F5DAE1DA257F6E0CEED375254A584018AF5
3B4BA4E62B01A66C1D47878AE77B
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 300 (seconds)
    TLS session ticket:
    0000 - f7 99 e8 a7 b9 53 f4 a3-74 3d c5 93 5c ee 8d 3b   .....S..t=..\..;
    0010 - bb 00 73 00 31 e6 dd 1e-73 38 4a 53 59 c9 27 fe   ..s.1...s8JSY.'.

    Start Time: 1775836329
    Timeout   : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
    Max Early Data: 0
```

Retorno a vir com código 0 (ok)



1.3

Editar o Virtual Host ´

- <VirtualHost *:443>
  ServerName localhost

```
    SSLEngine on

    SSLCertificateFile /etc/apache2/certs/fullchain.crt
    SSLCertificateKeyFile /etc/apache2/certs/server.key

    SSLCACertificateFile /etc/apache2/certs/rootCA.crt

    SSLVerifyClient require
    SSLVerifyDepth 2
</VirtualHost>
```

Teste com certificado de cliente

- curl -k https://localhost \
  --cert client.crt \
  --key client.key

Teste com openssl

- openssl s_client -connect localhost:443 \
  -cert client.crt \
  -key client.key \
  -CAfile ~/Desktop/rootCA/rootCA.crt

Resultado esperado (0) ok

Resultado:

```
SSL handshake has read 3123 bytes and written 373 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
---
Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
    Session-ID: B57F5CC00B8C2F1AC8516EA047E106453A4B4340B273A0AB5E99DD448CD4C4B3
    Session-ID-ctx:
    Resumption PSK: 3802978D285C22AC37D3ED9FF8107C75F56D77932C07442E312A1CDDE9010F8BEDE08C2
2977A427F47C1A32754D2989F
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 300 (seconds)
    TLS session ticket:
    0000 - 3a 66 e6 14 91 0b a1 95-dc 74 31 c3 05 06 54 50   :f.......t1...TP
    0010 - fc de 0b 79 44 aa 91 87-0a a6 b7 e6 e1 91 06 46   ...yD..........F

    Start Time: 1775841944
    Timeout   : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
    Max Early Data: 0
---
read R BLOCK
---
Post-Handshake New Session Ticket arrived:
```

Criar conteúdo privado

- sudo mkdir -p /var/www/html/private
  echo "PRIVATE CONTENT" | sudo tee /var/www/html/private/index.html

Restringir o acesso ao conteúdo

- <Directory "/var/www/html/private">
  SSLRequireSSL
  SSLVerifyClient require
  </Directory>

Teste com openssl

- openssl s_client -connect localhost:443 -CAfile ~/Desktop/rootCA/rootCA.crt

```
Abiigx/caCngjDpxtSOZL8MZuactYImWJjx4mzwDG4mruYnoTKCbTVhc6uNe2egd
1KRmi3GUxhz+NS3SSI4vQG4O222Dsh/JcurPCsQ4D88Wg6kCAwEAATANBgkqhkiG
9w0BAQsFAAOCAgEAasFrm6iwIhha8Wwxe7I4N16Mat5iwypPC6unM+SxttS7h8oW
4ZGiWfK9D2nXZZidpfL0SPfLNG3pjdeJvO2S9aUy6Ss+/24d94fy3w4qX48uO+2Z
Gk42wCJjnClEUm8bUaf6/bH4cLwXPK0Ofm4BT7jyQtIujEJj0StmXcXWfcOn7tzq
re2My0pVr+dIxs4cP17LqgYqHmsmyQ+YlWaN9RirrbE8BMzrKws7jVFLxcxs+Uze
z4o4SgG7TwW3mQVphXAozTd7ZPqAHe64q9au9j8+AtcB9Xf+AaALUPC7yrO84D7j
BjhMLJypC4ITPqSjpK9qlKxM/O42nuwNoFiCticL5cjc+TVYbxE3y2beGdmQsb2B
Jq0bA/fdlbjbhasnhgshwjNt7xHuCQuPlSKGtcIvcp5LJg0RyvROANw9ZqvD0UBd
/68utYvO9Sbb1ocvD+pmrTGVFROwWvWLhTSrQMGbJs8rGg1y1MXH0On20aa2P5XG
0ChrohnhQmLPvyvdtIRtGHvOdc6X3N9aVa9wPltWvsb9Cy8oTlBBifMWj4nBbiAr
mAtpWwWL+BHt6xZ5fBIaD3wC/uX39uBzCDXRBeu0ndzWK9zm/TcLq8FGwBYLdo54
Q8yA7S0Xm2MaSvAuOU3I5WNtngxQaej6/OwfWn4l6e85gW/wRMWZO5dw7tw=
-----END CERTIFICATE-----
subject=C = PT, ST = Porto, L = Porto, O = ISEP, OU = M1b, CN = localhost
issuer=C = PT, ST = Porto, L = Porto, O = ISEP, OU = m1b, CN = IntermediateCA
---
No client certificate CA names sent
Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed25519:E
d448:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+SHA384:
RSA-PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224
Shared Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:Ed
25519:Ed448:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA-PSS+SHA256:RSA-PSS+
SHA384:RSA-PSS+SHA512:RSA+SHA256:RSA+SHA384:RSA+SHA512
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 3190 bytes and written 403 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
```

PL5

Exercício 1

Desenvolvemos um conjunto de testes com o objetivo de analisar um ficheiro JPEG do ponto de vista forense, tentando identificar possíveis técnicas de esteganografia ou manipulação de dados. Não partimos do princípio de que existia informação escondida — procurámos validar isso de forma estruturada, cobrindo várias abordagens possíveis.

Começámos pelo mais básico: confirmar se o ficheiro é realmente aquilo que aparenta ser. No **Teste 1**, validámos os *magic bytes* para garantir que não estávamos perante um ficheiro disfarçado (por exemplo, um ZIP renomeado para JPEG). Em análise forense, não se pode confiar apenas na extensão.

No **Teste 2**, extraímos todas as strings ASCII do ficheiro. O objetivo foi identificar possíveis mensagens em texto plano escondidas no binário. Procurámos palavras-chave típicas como “flag”, “secret” ou “password”, comuns em cenários de ocultação simples.

No **Teste 3**, analisámos o marcador de fim de ficheiro do JPEG (FF D9). Qualquer conteúdo após este ponto não é visível na imagem, mas continua presente no ficheiro. Esta é uma técnica clássica de ocultação, por isso verificámos se existiam dados adicionais após o EOF.

No **Teste 4**, analisámos os comentários do JPEG (segmento FF FE). Este campo permite armazenar texto arbitrário e pode ser usado para esconder informação sem afetar a imagem.

No **Teste 5**, aprofundámos a análise dos metadados (segmentos APP como EXIF, XMP e IPTC). Estes campos podem conter informação relevante ou até dados manipulados com intenção de ocultação.

No **Teste 6**, procurámos strings em formato Base64. Muitas vezes, dados escondidos são codificados desta forma para parecerem aleatórios. Sempre que possível, tentámos decodificar e validar o conteúdo obtido.

No **Teste 7**, realizámos uma análise semelhante ao *binwalk*, procurando assinaturas de outros ficheiros dentro do JPEG. Isto permite detetar ficheiros embebidos, como ZIPs ou PDFs, anexados à imagem.

No **Teste 8**, analisámos a possibilidade de utilização de LSB (Least Significant Bit), uma técnica que altera bits menos significativos dos pixels para esconder informação sem impacto visual. Extraímos esses bits e tentámos reconstruir possíveis mensagens.

No **Teste 9**, aplicámos análise de entropia para avaliar o nível de aleatoriedade dos dados. Zonas com entropia elevada podem indicar dados comprimidos ou encriptados, enquanto valores baixos indicam padrões repetitivos.

No **Teste 10**, calculámos hashes criptográficos (MD5, SHA1, SHA256, SHA512) para garantir a integridade do ficheiro e permitir a sua identificação em bases de dados externas, como o VirusTotal.

No **Teste 11**, procurámos dados comprimidos escondidos (zlib/gzip). Uma técnica comum é comprimir a mensagem antes de a esconder, pelo que tentámos identificar e descomprimir blocos suspeitos.

Por fim, no **Teste 12**, realizámos uma análise estatística dos pixels (Chi-Square e distribuição par/ímpar). A esteganografia por LSB altera a distribuição natural dos valores dos pixels, e isso pode ser detetado matematicamente.

```
==================================================================
Resumo — RESULTADOS DOS 12 TESTES
==================================================================
  Assinatura do ficheiro (Magic Bytes)     SUSPEITO
  Strings ASCII suspeitas                  Normal
  Dados após marcador EOF (FF D9)          Normal
  Comentários JPEG (FF FE)                 Normal
  Segmentos APP/EXIF encontrados              1
  Strings Base64 decodificáveis            Normal
  Ficheiros embebidos (binwalk-like)       Normal
  Mensagem LSB nos pixels                  Normal
  Entropia (bits/byte)                        7.51446092484975
  Hash SHA256                                 84aafb1243fcbe21...
  Blocos zlib/gzip escondidos              Normal
  Análise estatística pixels (Chi²/RS)     Normal
```

Após obter os resultados, decidimos verificar o porquê de a assinatura/header do ficheiro com o objetivo de confirmar de forma técnica se o ficheiro era genuíno, perceber a sua estrutura interna e validar se existiam indícios de manipulação, corrupção ou utilização de segmentos não habituais.

Começámos por ler o ficheiro em modo binário e carregar todo o seu conteúdo para memória. A partir daí, executámos cinco verificações principais.

Começámos por visualizar os primeiros 64 bytes para validar a estrutura inicial. Depois, identificámos o tipo real do ficheiro através dos *magic bytes*, confirmando que corresponde a um JPEG/JFIF. Em seguida, verificámos os marcadores base do formato (FF D8 e segmentos APP), garantindo que o ficheiro segue a estrutura correta.

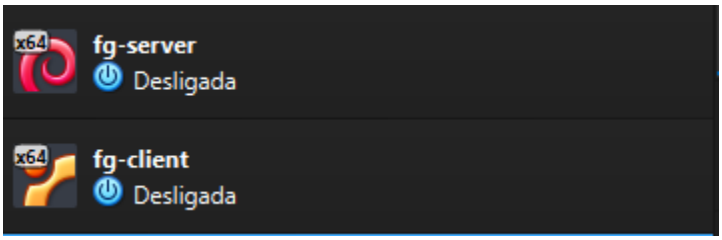
Posteriormente, percorremos todo o ficheiro para mapear os marcadores JPEG e validar a consistência interna. Por fim, testámos a abertura com a biblioteca PIL para confirmar que a imagem é interpretada corretamente.

```
1. Primeiros 64 bytes (hex + ASCII):
  0000:  FF D8 FF E0 00 10 4A 46 49 46 00 01 01 00 00 01  ......JFIF......
  0010:  00 01 00 00 FF DB 00 43 00 09 06 07 08 07 06 09  .......C........
  0020:  08 07 08 0A 0A 09 0B 0D 16 0F 0D 0C 0C 0D 1B 14  ................
  0030:  15 10 16 20 1D 22 22 20 1D 1F 1F 24 28 34 2C 24  ... ."" ...$(4,$

2. Identificação detalhada do tipo:
  Tipo identificado: JPEG/JFIF
    Descrição: Padrão normal - câmaras, web
    Magic bytes: FFD8FFE0

3. Verificação de marcadores JPEG:
  Começa com FF D8 — é JPEG válido na base
  Byte 3: 0xFF  Byte 4: 0xE0
  APP0 (JFIF) — câmara/web padrão

4. Mapa completo de marcadores JPEG:
  Pos      0: FF D8  SOI  (Start of Image)
  Pos      2: FF E0  APP0 (JFIF)  [16 bytes]
  Pos     20: FF DB  DQT  (Quantization Table)  [67 bytes]
  Pos     89: FF DB  DQT  (Quantization Table)  [67 bytes]
  Pos    158: FF C0  SOF0 (Baseline DCT)  [17 bytes]
  Pos    177: FF C4  DHT  (Huffman Table)  [31 bytes]
  Pos    210: FF C4  DHT  (Huffman Table)  [181 bytes]
  Pos    393: FF C4  DHT  (Huffman Table)  [31 bytes]
  Pos    426: FF C4  DHT  (Huffman Table)  [181 bytes]
  Pos    609: FF DA  SOS  (Start of Scan - dados de imagem)  [12 bytes]
  Pos   4949: FF D9  EOI  (End of Image)

5. Tentativa de abertura como imagem (PIL):
  Abre com PIL sem erro
  Formato PIL: JPEG
  Modo:        RGB
  Dimensões:   204 x 204 px
  Info EXIF:   ['jfif', 'jfif_version', 'jfif_unit', 'jfif_density']
```

O ficheiro é um JPEG/JFIF válido, com estrutura normal e sem anomalias no header. Não foram identificados indícios de manipulação ou dados suspeitos ao nível do cabeçalho.

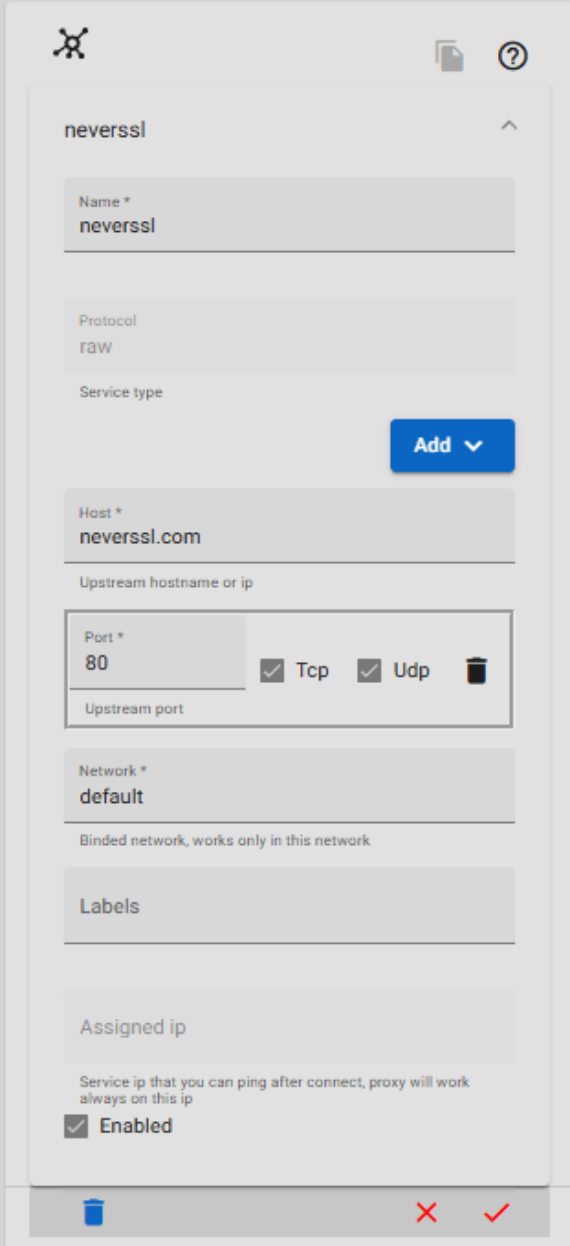
Exercício 2

Criação das Virtual Machines



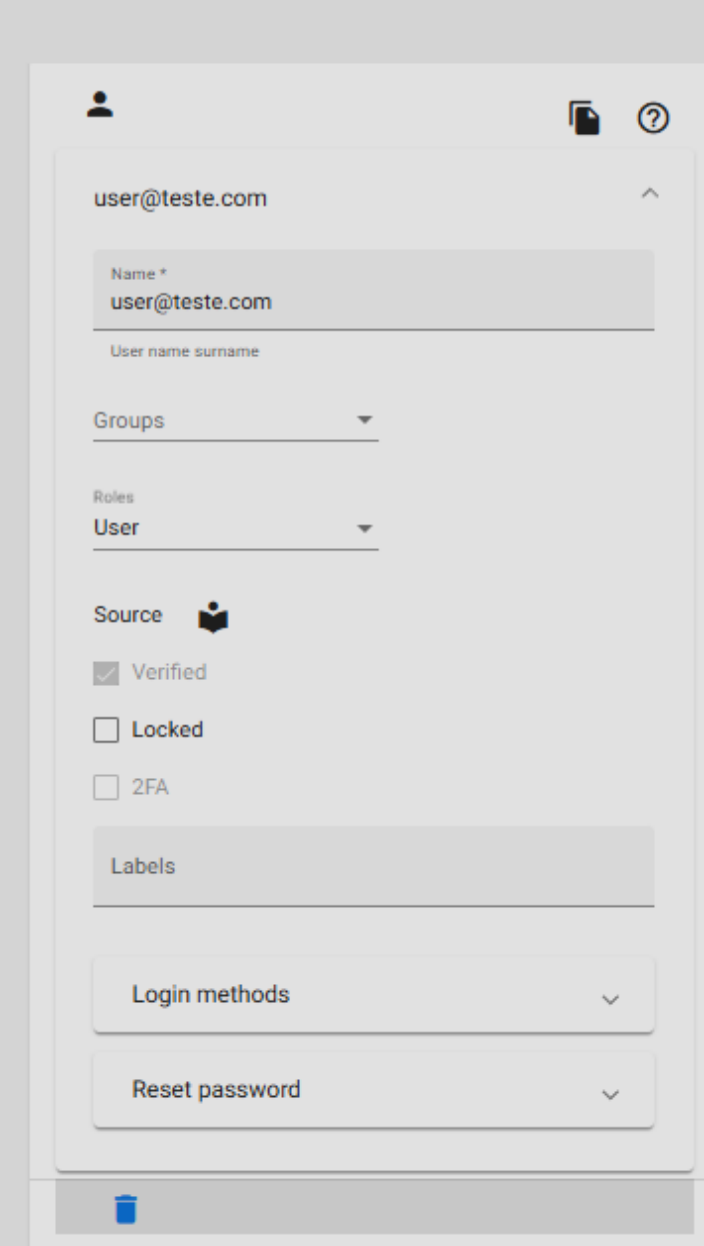
Configuração de Serviços

Foi criado um serviço para acesso ao domínio neverssl.com.



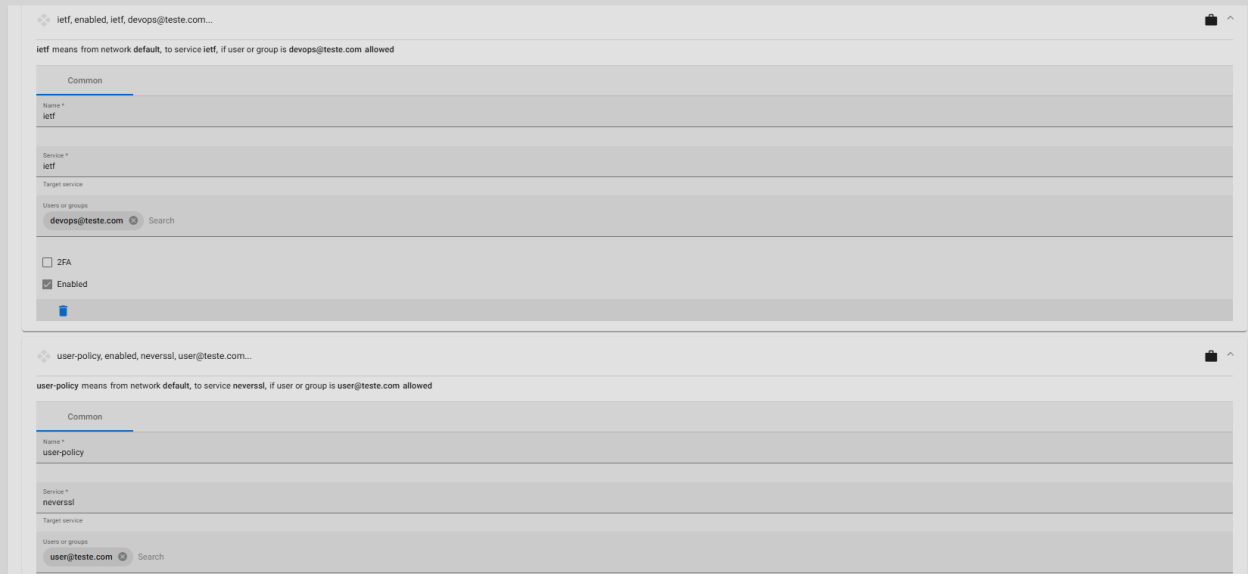
Criação de Utilizadores

Foi criado um utilizador para testes de acesso com permissões básicas.



Políticas de Autorização

Foram definidas políticas para controlar o acesso aos serviços. O utilizador user@teste.com tem acesso ao serviço neverssl, enquanto devops possui políticas distintas.



Estado do Sistema

O sistema foi validado através da verificação do estado do serviço FerrumGate e dos containers Docker.

```
○ ferrumgate.service -  service with docker compose
     Loaded: loaded (/etc/systemd/system/ferrumgate.service; enabled; preset: enabled)
     Active: inactive (dead)
```

Conclusão

A implementação do FerrumGate permitiu validar o modelo Zero Trust, demonstrando que o acesso aos serviços é controlado através de autenticação e políticas. Este modelo oferece maior segurança comparado com abordagens tradicionais.

1. Para permitir acesso remoto a serviços internos, foram criadas políticas baseadas na identidade e segurança do dispositivo.
2. Sim, é possível aumentar a segurança aplicando 2FA, restrições por função e validação do estado do dispositivo.
3. O 2FA aumenta a segurança mas pode dificultar a usabilidade.
4. Permite aplicar políticas mais rigorosas a dispositivos pessoais, garantindo conformidade.
5. O NAC baseia-se na rede, enquanto o Zero Trust valida continuamente identidade e acesso.

Exercício 3

A utilização da ferramenta **O&O ShutUp10++** permitiu aplicar, de forma prática, os princípios definidos no **CIS Controls Privacy Guide**, cujo principal objetivo é integrar medidas de cibersegurança com a proteção da privacidade dos utilizadores. O guia defende a redução da recolha de dados desnecessários, a limitação de acessos indevidos e a adoção de configurações seguras por defeito, de forma a diminuir riscos para utilizadores e organizações.

No contexto deste exercício, as alterações realizadas no sistema Windows seguiram esses princípios. Em primeiro lugar, foi aplicada a **minimização de dados**, através da desativação de funcionalidades de telemetria, recolha de dados de diagnóstico, publicidade personalizada, sugestões automáticas e sincronização excessiva de informações. Desta forma, reduziu-se a quantidade de dados pessoais enviados para serviços externos.

Em segundo lugar, foi aplicado o princípio do **menor privilégio**, restringindo permissões de aplicações para acesso a recursos sensíveis como localização, microfone, câmara, contactos, calendário, ficheiros e histórico de atividade. Esta medida reduz o risco de exposição indevida de dados pessoais e limita o impacto de aplicações maliciosas ou excessivamente permissivas.

Por fim, foi reforçada a **configuração segura por defeito**, ao aplicar as recomendações seguras sugeridas pela ferramenta. Estas configurações incluíram também a limitação de funcionalidades de tracking no Microsoft Edge, Microsoft Office, Cortana e Copilot, reduzindo a superfície de exposição do sistema.

Estas medidas estão alinhadas com o **CIS Control 4 – Secure Configuration of Enterprise Assets and Software**, que recomenda a adoção de configurações seguras em sistemas operativos e dispositivos de utilizador, de modo a reforçar simultaneamente a segurança e a privacidade.

Assim, conclui-se que a utilização do O&O ShutUp10++ permitiu melhorar significativamente a privacidade do sistema, proporcionando maior controlo sobre os dados pessoais, reduzindo a recolha de informação desnecessária e tornando o sistema mais seguro e menos intrusivo para o utilizador.

Exercício 4

- Extract the archive

- Obtain superSuper.img
- Run: file superSuper.img
- Result:
  - LUKS encrypted file (version 2)
- Conclusion:
  - Disk protected with LUKS2

```
deus@Ubuntu-Lab:~/Desktop$ file superSuper.img
superSuper.img: LUKS encrypted file, ver 2, header size 16384, ID 3, algo sha256
, salt 0xfb7d8e4057e473e7..., UUID: c88b31c7-6fbb-41ce-a112-9e9a1f1d64c9, crc 0x
3fb375c4d7397b4a..., at 0x1000 {"keyslots":{"0":{"type":"luks2","key_size":64,"a
f":{"type":"luks1","stripes":4000,"hash":"sha256"},"area":{"type":"raw","offse
deus@Ubuntu-Lab:~/Desktop$
```

- luks2john → not compatible (only LUKS1)
- Chosen method:
  - cryptsetup + dictionary attack
- Reason:
  - Compatible with LUKS2
  - Required use of rockyou

Obtain Rockyou Wordlist

- Download wordlist: wget https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
- Verify: head rockyou.txt and wc -l rockyou.txt

Prepare Brute Force Script

- Create script file: nano brute.sh
- Script content:

Make executable: chmod +x brute.sh

```bash
#!/bin/bash

WORDLIST=~/Desktop/wordlists/rockyou.txt
IMAGE=~/Desktop/superSuper.img
MAPPER_NAME=decrypted_disk
LOG=output.log

COUNT=0

echo "Starting brute force..." | tee $LOG

while IFS= read -r pass; do
  COUNT=$((COUNT+1))

  # Mostrar progresso a cada 1000 tentativas
  if (( COUNT % 1000 == 0 )); then
    echo "Tried $COUNT passwords..." | tee -a $LOG
  fi

  echo "Trying: $pass" >> $LOG

  echo -n "$pass" | sudo cryptsetup luksOpen "$IMAGE" "$MAPPER_NAME" --key-file=- 2>/dev/null

  if [ $? -eq 0 ]; then
    echo "PASSWORD FOUND: $pass" | tee -a $LOG
    echo "$pass" > found.txt
    break
  fi
```

Run script: ./brute.sh

- The script:
  - Reads each password from rockyou
  - Attempts to unlock the disk
  - Stops when correct password is found

Monitor Progress

- View live output: tail -f output.log
- Check if password found: grep “PASSWORD FOUND” output.log
- Confirm disk unlocked: ls /dev/mapper/

After success, device created:

➔ /dev/mapper/decrypted_disk

- sudo mkdir /mnt/decrypted
- sudo mount /dev/mapper/decrypted_disk /mnt/decrypted

<u>10. Retrieve Flag</u>

- ls /mnt/decrypted

- find /mnt/decrypted -type f
- cat /mnt/decrypted/*

Distribuição de Tarefas

	Exercício	Aluno
PL1	1	1211882
	2	1250520
PL2	1 (Obrigatório para grupos de 4)	1250494
	2	1250494
	3	1211882
	4	1200882
PL3	1	1211882
	2	1200882
	2 (Obrigatório para grupos de 4)	1250520
	3	1250494
PL4	1	1200882
	2	1200882
	3	1200882
PL5	1	1250494
	2	1250520
	3	1211882
	4 (Obrigatório para grupos de 4)	1200882